Autómata de una máquina expendedora

Caso práctico

El instituto quiere poner una máquina de bebidas en la cafetería y el profesor de la asignatura le pide a **Carlos** que simule su programa en PHP antes de comprarla. **Juan** le cuenta lo que no se ve desde fuera: la máquina tiene que saber dar el **cambio** con las monedas que tiene, no solo calcularlo. —Si no tiene monedas de 10 céntimos, ese café no lo puede vender.

1.- Enunciado

Programa en PHP una **máquina expendedora** (sin sesiones: el estado de la máquina viaja en un campo oculto):

- Productos: agua (0,80 €, 3 unidades), café (1,10 €, 2), zumo (1,50 €, **agotado**) y bocadillo (2,35 €, 1). Monedas aceptadas: 5, 10, 20, 50 céntimos, 1 € y 2 €. El cajón para el cambio empieza con una moneda de 50, una de 20, dos de 10 y dos de 5 céntimos.
- Se **elige** un producto (si está agotado o no existe, se avisa). Después se van **introduciendo monedas** hasta llegar al precio; una moneda no aceptada se devuelve.
- Al llegar al precio, si se puede dar el **cambio exacto** con las monedas del cajón (incluidas las que se acaban de introducir), se entrega el producto y el cambio; si no, se **devuelve todo el dinero** ("No hay cambio suficiente").
- *Cancelar* devuelve lo introducido. En las pantallas de entrega y de devolución hay que pulsar *Recoger*.
- **Mantenimiento**: con la clave `2468` se entra en un panel que muestra existencias y monedas y permite **reponer** (5 unidades de cada producto y 5 monedas de cada tipo).
- Si el campo oculto llega manipulado, la máquina se **reinicia**.

2.- Análisis

- **Eventos entrantes.** Nueve. *Elegir* y *Moneda* llevan un dato (qué producto, qué moneda): van en el valor del botón, `accion=elegir:cafe`, `accion=moneda:200`.
- **Estados.** Siete: esperando selección, esperando el pago, producto entregado, dinero devuelto, pidiendo la clave y mantenimiento (más ST0). "Entregado" y "devuelto" son estados distintos porque muestran cosas distintas, aunque desde los dos solo se pueda *Recoger*.
- **Variables de estado.** La máquina entera en JSON: existencias, monedas del cajón, producto elegido y monedas introducidas.
- **Predicados.** El más interesante es **PL5: se puede dar el cambio**. No basta con restar: hay que **buscar** una combinación con las monedas que hay. Coger siempre la moneda más grande no funciona con monedas limitadas (60 céntimos con una de 50 y tres de 20: tras la de 50 faltan 10 y no hay; con tres de 20 sí sale), así que la búsqueda retrocede si no encuentra.
- **PL0 en todas las transiciones** (menos T1): si el JSON no es coherente (faltan productos, cantidades negativas, monedas que no existen), la máquina se reinicia.

3.- Diagrama de estado-transición ampliado

La caja discontinua "ST3, ST4" representa los dos estados: *Recoger* hace lo mismo en los dos. La flecha "cualquiera, !PL0" desde STx resume el primer camino de todas las transiciones T2–T9.

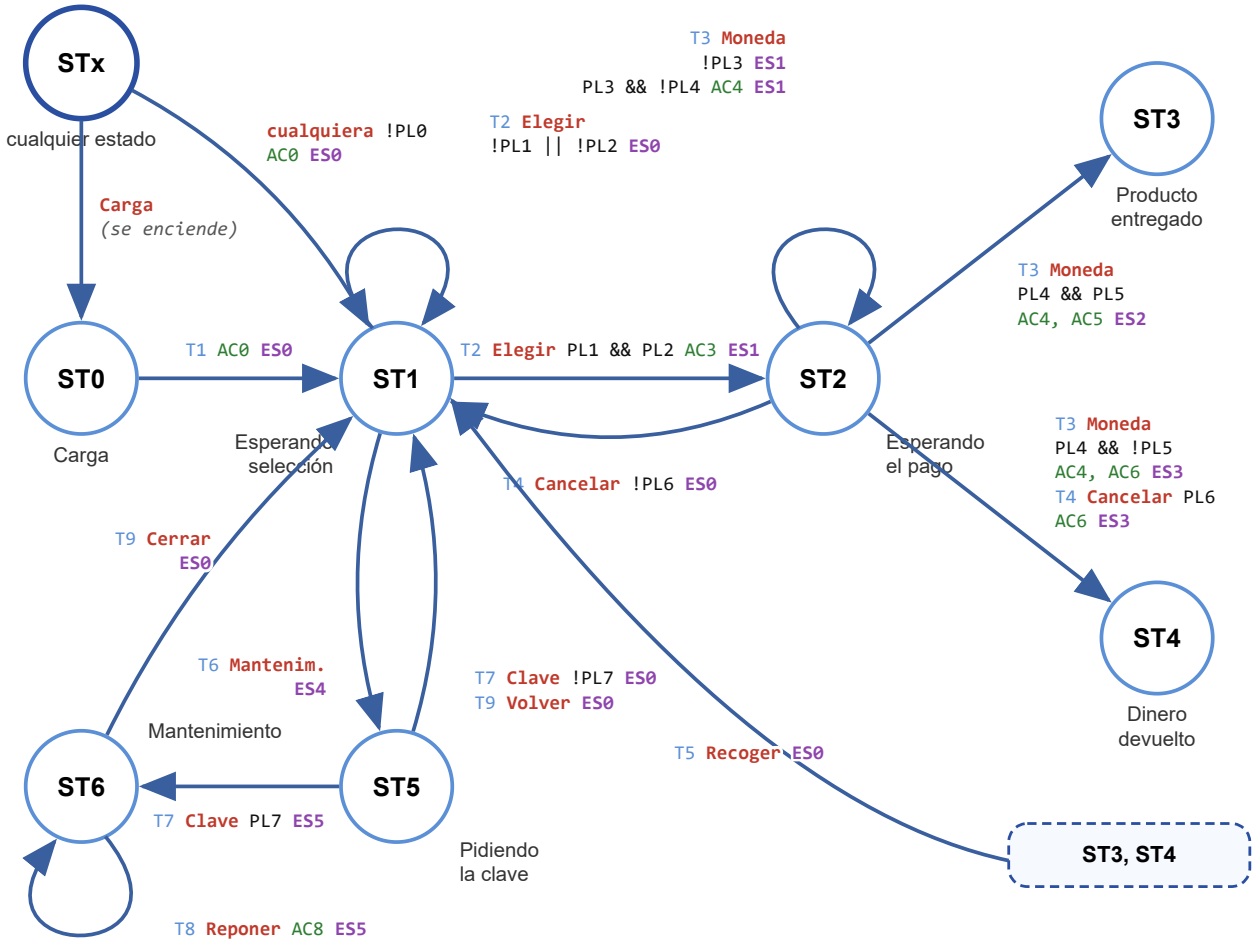


Figura 1. Diagrama de la máquina expendedora

4.- Autómata

4.1.- Tabla de descriptores

Tipo	Abreviatura	Nombre	Significado
Evento entrante	EE0	Carga	se enciende la máquina (página sin parámetros)
Evento entrante	EE1	Elegir	un botón de producto (`elegir:codigo`)
Evento entrante	EE2	Moneda	un botón de moneda (`moneda:valor`)
Evento entrante	EE3	Cancelar	en el pago: devolver; en la clave: volver
Evento entrante	EE4	Recoger	recoger el producto o el dinero
Evento entrante	EE5	Mantenimiento	ir a pedir la clave
Evento entrante	EE6	Clave	enviar la clave
Evento entrante	EE7, EE8	Reponer, Cerrar	botones del panel
Evento saliente	ES0	seleccion	productos (los agotados marcados) y un aviso si lo hay
Evento saliente	ES1	pago	precio, crédito, lo que falta, botones de monedas
Evento saliente	ES2	entregado	"Recoja su ..." y el cambio
Evento saliente	ES3	devuelto	motivo y monedas devueltas
Evento saliente	ES4	pedirClave	
Evento saliente	ES5	panel	existencias y monedas del cajón
Estados	ST0–ST6	—	Carga, EsperandoSeleccion, EsperandoPago, Entregado, Devuelto, PidiendoClave, Mantenimiento
Variable de estado	VE0	`$estado`	
Variable de estado	VE1	`$m`	JSON: existencias, cajón, producto, crédito
Variable	VE2	bandeja	monedas o producto que recoge el cliente (no viaja)
Predicado	PL0	máquina coherente	el JSON tiene la forma y los valores esperados

Tipo	Abreviatura	Nombre	Significado
Predicado	PL1	el producto existe	
Predicado	PL2	hay existencias	
Predicado	PL3	moneda aceptada	5, 10, 20, 50, 100 o 200
Predicado	PL4	llega al precio	crédito + moneda ≥ precio
Predicado	PL5	hay cambio	`buscarCambio(crédito - precio, cajón + crédito) !== null`
Predicado	PL6	hay crédito	se ha introducido alguna moneda
Predicado	PL7	clave correcta	`hash_equals('2468', clave)`
Acción	AC0	máquina nueva	existencias y cajón iniciales
Acción	AC1	recuperar la máquina	decodificar y comprobar (PL0)
Acción	AC2	leer el dato	producto o moneda del valor del botón
Acción	AC3	elegir	producto = elegido; crédito vacío
Acción	AC4	añadir la moneda	al crédito
Acción	AC5	cobrar	monedas al cajón, sacar el cambio, existencias − 1
Acción	AC6	devolver	el crédito a la bandeja
Acción	AC7	vaciar la bandeja	
Acción	AC8	reponer	5 de cada producto y 5 de cada moneda
Acción	AC9	guardar	la máquina en el campo oculto (en todas las respuestas)

4.2.- Tabla de eventos-estados ampliada

	Carga	Elegir	Moneda	Cancelar	Recoger	Manten.	Clave	Reponer	Cerrar
ST0	T1	T1	T1	T1	T1	T1	T1	T1	T1
ST1	T1	T2	---	---	---	T6	---	---	---
ST2	T1	---	T3	T4	---	---	---	---	---
ST3	T1	---	---	---	T5	---	---	---	---
ST4	T1	---	---	---	T5	---	---	---	---
ST5	T1	---	---	T9	---	---	T7	---	---
ST6	T1	---	---	---	---	---	---	T8	T9

4.3.- Tabla de transiciones

#	AEs antes	PLs	Nuevo ST	AEs después	ESs
T1	---	---	ST1	AC0	ES0 [AC9]
T2–T9	AC1	`!PL0`	ST1	AC0	ES0 "se ha reiniciado" [AC9]
T2	AC1, AC2	`PL0 && !PL1`	ST1	---	ES0 "no existe" [AC9]
T2	AC1, AC2	`PL0 && PL1 && !PL2`	ST1	---	ES0 "agotado" [AC9]
T2	AC1, AC2	`PL0 && PL1 && PL2`	ST2	AC3	ES1 [AC9]
T3	AC1, AC2	`PL0 && !PL3`	ST2	---	ES1 "moneda no aceptada" [AC9]
T3	AC1, AC2	`PL0 && PL3 && !PL4`	ST2	AC4	ES1 [AC9]
T3	AC1, AC2	`PL0 && PL3 && PL4 && PL5`	ST3	AC4, AC5	ES2 [AC9]
T3	AC1, AC2	`PL0 && PL3 && PL4 && !PL5`	ST4	AC4, AC6	ES3 "no hay cambio" [AC9]
T4	AC1	`PL0 && PL6`	ST4	AC6	ES3 "cancelada" [AC9]
T4	AC1	`PL0 && !PL6`	ST1	---	ES0 [AC9]
T5	AC1	`PL0`	ST1	AC7	ES0 [AC9]
T6	AC1	`PL0`	ST5	---	ES4 [AC9]
T7	AC1	`PL0 && PL7`	ST6	---	ES5 [AC9]
T7	AC1	`PL0 && !PL7`	ST1	---	ES0 "clave incorrecta" [AC9]
T8	AC1	`PL0`	ST6	AC8	ES5 [AC9]
T9	AC1	`PL0`	ST1	---	ES0 [AC9]

4.4.- Comprobación de predicados (T3, con PL0 cierto)

PL3	PL4	PL5	moneda no aceptada	falta dinero	entregar	devolver
0	x	x	1	0	0	0
1	0	x	0	1	0	0
1	1	1	0	0	1	0
1	1	0	0	0	0	1

PL5 solo se calcula si PL4 es cierto (si no se llega al precio no hay cambio que dar).

5.- Del autómata al código

Fragmento de `solucion/index.php`. Observa que PL3 y PL4 se calculan **antes** de cambiar nada, y que AC4 (añadir la moneda) se hace en tres de los cuatro caminos:

```
case 3:                                                 // ---- T3 (ST2, Moneda)
    $moneda = filter_var($dato, FILTER_VALIDATE_INT);  // AC2: leer la moneda
    $precio = PRODUCTOS[$m['producto']][1];
    $pl3 = in_array($moneda, MONEDAS, true);           // PL3: moneda aceptada
    $creditoTotal = array_sum($m['credito']) + ($pl3 ? $moneda : 0);
    $pl4 = $pl3 && $creditoTotal >= $precio;           // PL4: llega al precio
    if (!$pl3) {                                       // PL0 && !PL3
        $estado = ST2;
        es1($m, 'Moneda no aceptada: se la devolvemos.');
    } elseif (!$pl4) {                                 // PL0 && PL3 && !PL4
        $estado = ST2;
        $m['credito'][] = $moneda;                     // AC4
        es1($m);
    } else {
        $m['credito'][] = $moneda;                     // AC4
        $cajon = $m['cajon'];                          // las monedas introducidas también
sirven para el cambio
        foreach ($m['credito'] as $c) {
            $cajon[$c]++;
        }
        $cambio = buscarCambio($creditoTotal - $precio, $cajon);
        if ($cambio !== null) {                        // PL0 && PL3 && PL4 && PL5
            $estado = ST3;
            // AC5: cobrar, entregar y dar el cambio (ver el fichero)
            es2($m);
        } else {                                       // PL0 && PL3 && PL4 && !PL5
            $estado = ST4;
            $m['bandeja'] = $m['credito'];             // AC6: devolver el crédito
            $m['producto'] = null;
            $m['credito'] = [];
            es3($m, 'No hay cambio suficiente');
        }
    }
    break;
```

La búsqueda del cambio (PL5), recursiva y con vuelta atrás:

```php
function buscarCambio(int $cambio, array $disponibles, int $desde = 0): ?array
{
    if ($cambio === 0) {
        return [];
    }
    for ($i = $desde; $i < count(MONEDAS); $i++) {
        $moneda = MONEDAS[$i];
        if ($moneda <= $cambio && $disponibles[$moneda] > 0) {
            $disponibles[$moneda]--;
            $resto = buscarCambio($cambio - $moneda, $disponibles, $i);
            if ($resto !== null) {
                return [$moneda, ...$resto];
            }
            $disponibles[$moneda]++;
        }
    }
    return null;
}
// buscarCambio(60, [200 => 0, 100 => 0, 50 => 1, 20 => 3, 10 => 0, 5 => 0])  ->  [20, 20, 20]
```

6.- Pruebas

30 peticiones; algunos caminos:

Prueba	Camino	Resultado comprobado
zumo; "pizza"	T2 `!PL2`; T2 `!PL1`	"agotado"; "no existe"
café, 3 céntimos, 1 €, 20 céntimos	T2, T3 `!PL3`, `!PL4`, entregar	cambio 0,10 €
máquina nueva, agua con 2 €	T3 `!PL5`	1,20 € de cambio imposible (el cajón solo suma 1,00 €): se devuelven los 2 €
agua, 50 céntimos, Cancelar	T4 `PL6`	devuelve 0,50 €
bocadillo, Cancelar	T4 `!PL6`	vuelve a la selección
clave 0000; clave 2468; Reponer; Cerrar	T7 `!PL7`, T7 `PL7`, T8, T9	aviso; panel; zumo con 5 unidades
zumo con 2 € tras reponer	entregar	cambio de 0,50 €
JSON de la máquina cambiado	`!PL0`	"se ha reiniciado"
Moneda en ST3	`---` → T1	selección

7.- Para practicar

- Añade **"Elegir otro producto"** mientras se paga, conservando el crédito. ¿Qué celda de la tabla deja de ser `---`? ¿Qué pasa si el crédito ya supera el precio del nuevo producto?

- Muestra en la selección qué productos **no se podrían pagar con una moneda de 2 €** por falta de cambio. ¿Es un predicado nuevo, una acción o parte de un evento saliente?
